

Q1 Employee Payroll and Human Resource Management System

50 marks

A multinational software company employs thousands of professionals across various development centers. Employee information is collected during recruitment and transferred from different regional offices. Due to inconsistent data entry practices, employee names, email addresses, and department names are stored in different formats. Duplicate employee IDs have also been identified. The Human Resource department requires a Java application capable of validating employee information and organizing payroll records efficiently using Collection Framework classes.

Develop a Java application that performs the following tasks:

- Read Employee ID, Name, Department, Designation, Salary, Email ID, and Branch.
- Use String Handling methods to standardize names and validate official email IDs.
- Store employee records in an ArrayList.
- Remove duplicate Employee IDs using a HashSet.
- Map Employee IDs with Salary details using a HashMap.
- Traverse employee records using Iterator.
- Implement all collections using Generics.
- Generate department-wise salary reports and explain the role of each Collection Interface in the application.

Your Code

```
1 import java.util.*;
2 public class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         ArrayList<Employee> emps=new ArrayList<Employee>();
6         //(a) READ EMPLOYEE ID,NAME,DEPARTMENT,DESIGNATION,SALARY,EMAIL ID,BRANCH
7         System.out.print("enter no of employees: ");
8         int n=sc.nextInt();
9         sc.nextLine();
10        for(int i=0;i<n;i++){
11            String id=sc.nextLine();
12            String name=sc.nextLine();
13            String dept=sc.nextLine();
14            String desg=sc.nextLine();
15            double sal=sc.nextDouble();
16            sc.nextLine();
17            String em=sc.nextLine();
18            String br=sc.nextLine();
19            //(B)STRNG HANDLING: STANDARDISE NAMES AND VALIDATE OFFICIAL EMAIL IDs
20            name= name.trim();
21            String[] wds=name.toLowerCase().split(" ");
22            String sdname=" ";
23            for(String word : wds){
24                if(word.length()>0){
25                    sdname=sdname+Character.toUpperCase(word.charAt(0));
26                }
27            }
28            sdname=sdname.trim();
29            em=em.trim().toLowerCase();
30            boolean vemail=em.endsWith("@company.com");
31            if(!vemail){
32                System.out.println("invalid official email ID");
33                continue;
34            }
35            Employee emp =new Employee(id,sdname,dept,desg,sal,em,br);
36            //(c)STORE EMPLOYEE RECORDS IN ARRAYLIST
37            emps.add(emp);
38        }
39        //(d)REMOVE DUPLICATE EMPLOYEE IDs USING HASHSET
40        HashSet<String> emids=new HashSet<String>();
41        ArrayList<Employee> ugem=new ArrayList<Employee>();
42        for(Employee emp:emps){
```

Input

```
4
101
emily
IT
```

Output

```
enter no of employees:
EMPLOYEE RECORDS
101 Emily IT developer 60000.0
emily101@company.com chennai
102 John HR manager 70000.0
john@company.com bangalore
```

```

40     HashSet<String> emids=new HashSet<String>();
41     ArrayList<Employee> ugem=new ArrayList<Employee>();
42     for(Employee emp:emps){
43         if(emids.add(emp.id)){
44             ugem.add(emp);
45         }
46         else{
47             System.out.println("duplicate employee id removes: "+emp.id);
48         }
49     }
50     // (e) MAP EMPLOYEE IDs WITH SALARY USING HASHMAP
51     HashMap<String,Double> smap=new HashMap<String,Double>();
52     for(Employee emp:ugem){
53         smap.put(emp.id,emp.sal);
54     }
55     // (f) TRAVERSE EMPLOYEE RECORDS USING ITERATOR
56     System.out.println("\nEMPLOYEE RECORDS");
57     Iterator<Employee> it=ugem.iterator();
58     while(it.hasNext()){
59         Employee emp=it.next();
60         System.out.println(emp.id+" "+emp.name+" "+emp.dept+" "+emp.sal);
61     }
62     // (g) IMPLEMENT COLLECTIONS USING GENERICS
63     System.out.println("\nCOLLECTION USING GENERICS:");
64     System.out.println("ArrayList: "+ugem.size()+" records");
65     System.out.println("HashSet: "+emids.size()+" unique IDs");
66     System.out.println("HashMap: "+smap.size()+" salary records");
67     // (h) DEPARTMENT-WISE SALARY REPORT
68     HashMap<String,Double> deptsal=new HashMap<String,Double>();
69     for(Employee emp : ugem){
70         String de=emp.dept;
71         if(deptsal.containsKey(de)){
72             double tot=deptsal.get(de)+emp.sal;
73             deptsal.put(de,tot);
74         }
75         else{
76             deptsal.put(de,emp.sal);
77         }
78     }
79     System.out.println("\nDEPARTMENT-WISE SALARY REPORT:");
80     Iterator<Map.Entry<String,Double>> deptit=deptsal.entrySet().iterator();
81     while(deptit.hasNext()){
82         Map.Entry<String,Double> entry=deptit.next();
83         System.out.println(entry.getKey()+" : "+entry.getValue());
84     }

```

```
84     }
85     //DISPLAY EMPLOYEE ID AND SALARY
86     System.out.println("\nEMPLOYEE SALARY MAP:");
87     Iterator<Map.Entry<String,Double>> salit=smap.entrySet().itera
88     while(salit.hasNext()){
89         Map.Entry<String,Double> entry=salit.next();
90         System.out.println(entry.getKey()+" : "+entry.getValue());
91     }
92     sc.close();
93 }
94 }
95 class Employee{
96     String id,name,dept,desg,em,br;
97     double sal;
98     Employee(String id,String name,String dept,String desg,double sal,
99         this.id=id;
100         this.name=name;
101         this.dept=dept;
102         this.desg=desg;
103         this.sal=sal;
104         this.em=em;
105         this.br=br;
106     }
107 }
```